

Iterator

Prof. Igor Avila Pereira
igor.pereira@riogrande.ifrs.edu.br

Instituto Federal de Educação, Ciência e Tecnologia do Rio Grande do Sul (IFRS)
Câmpus Rio Grande
Divisão de Computação

Agenda

- 1 Introdução
- 2 Exemplo
- 3 Conheça o Padrão Iterator
- 4 Definindo o Padrão Iterator
- 5 Trabalho
 - Examinando o Menu do Café

Introdução

Há muitas maneiras de agrupar objetos em uma coleção..

Ex:

- Matriz
- Pilha
- Lista
- Hashtable (Tabela Mista)

Cada uma destas opções tem suas vantagens e desvantagens. Entretanto, em algum ponto o seu cliente terá que iterar com esses objetos: Quando isso acontecer, você lhe mostrará a sua implementação?

Certamente que não!! Isto seria muito pouco profissional.

Fusão do Restaurante Objetolândia como Panquecaria Objetolândia

Mostraremos como você pode permitir que seus clientes realizem a iteração com seus objetos sem ter a mínima ideia de como eles são armazenados

Esta é uma ótima notícia!

Agora podemos comer aquelas panquecas deliciosas da Panquecaria no café da manhã e saborear aquelas fartos almoços do Restaurante em um único lugar.

Mas parece haver um pequeno problema.....

Fusão do Restaurante Objetolândia como Panquecaria Objetolândia

Eles querem usar o cardápio da minha Panquecaria no café da manhã e o cardápio do Restaurante no almoço. Concordamos em implementar esses itens no menu...

Lou →



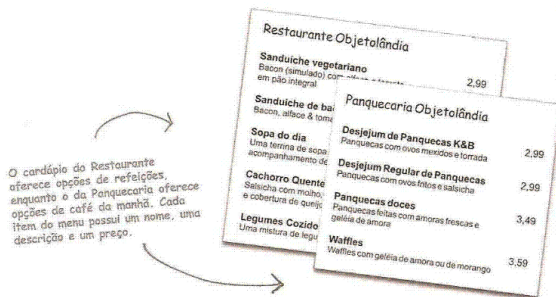
Mel →



... mas não conseguimos chegar a um acordo sobre como fazer essa implementação. Aquele sujeito ali usava um ArrayList para armazenar seus itens de menu, enquanto eu usava uma matriz. Nenhum de nós concorda em mudar a sua implementação... Temos um grande volume de código dependendo delas

Examinando os Itens do Menu

Pelo menos Lou e Mel concordam com a implementação dos itens de menu em MenuItems. Vamos examinar os itens de cada menu e a forma como estes foram implementados.

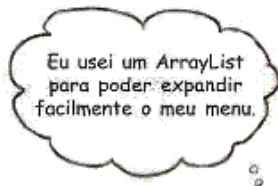


Examinando os Itens do Menu

```
1 public class MenuItem {
2     String name;
3     String description;
4     double price;
5     boolean vegetarian;
6
7     public MenuItem(String name, String description,
8         boolean vegetarian, double price) {
9         this.name = name;
10        this.description = description;
11        this.price = price;
12        this.vegetarian = vegetarian;
13    }
14    // getters e setters
15 }
```

As Implementações de Menus do Lou e Mel

Agora vamos ver por que Lou e Mel estão discutindo tanto. Ambos investiram muito tempo e trabalho para codificar a maneira como seus itens de menu são armazenados, bem como todos os outros trechos de código que dependem dessas informações.



As Implementações de Menus do Lou e Mel

Esta é a implementação de Lou
para o menu da sua
Panquecaria.

```
public class PancakeHouseMenu {  
    ArrayList menuItems;  
  
    public PancakeHouseMenu() {  
        menuItems = new ArrayList();  
  
        addItem("K&B's Pancake Breakfast",  
                "Pancakes with scrambled eggs, and toast",  
                true,
```

Lou está usando um ArrayList para
armazenar seus itens de menu.

As Implementações de Menus do Lou e Mel

```
        2.99);  
  
addItem("Regular Pancake Breakfast",  
        "Pancakes with fried eggs, sausage",  
        false,  
        2.99);  
  
addItem("Blueberry Pancakes",  
        "Pancakes made with fresh blueberries",  
        true,  
        3.49);  
  
addItem("Waffles",  
        "Waffles with your choice of blueberries or strawberries",  
        true,  
        3.59);  
  
}
```

Os itens de menu são acrescentados ao ArrayList aqui, no construtor.

Cada item possui um nome, uma descrição, uma indicação de se é ou não vegetariano e um preço.

As Implementações de Menus do Lou e Mel

```
public void addItem(String name, String description,  
                    boolean vegetarian, double price)
```

```
{
```

```
    MenuItem menuItem = new MenuItem(name, description, vegetarian, price);  
    menuItems.add(menuItem);
```

```
}
```

```
public ArrayList getMenuItems() {
```

```
    return menuItems;
```

```
}
```

```
// outros métodos de menu aqui
```

```
}
```

Para inserir um item de menu, Lou cria um novo objeto `MenuItem`, passando a ele todos os argumentos, e em seguida acrescenta esse objeto ao `ArrayList`.

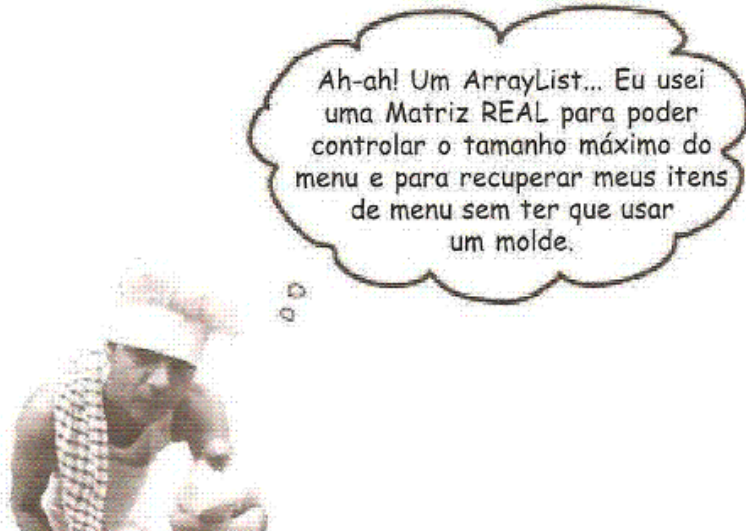
O método `getMenuItems()` retorna a lista de itens do menu.

Lou usa também outros blocos de código que dependem da implementação de `ArrayList`. Ele não quer ter que reescrever todo esse código!

As Implementações de Menus do Lou e Mel

- Lou está usando um `ArrayList` para armazenar seus itens de menu
- Os itens de menu são acrescentados ao `ArrayList` no construtor
- Cada item possui um nome, uma descrição, uma indicação de se é ou não vegetariano e um preço
- Para inserir um item de menu, Lou cria um novo objeto `Menuitem`, passando a ela todos os argumentos, e em seguida, acrescenta esse objeto ao `ArrayList`

As Implementações de Menus do Lou e Mel



Ah-ah! Um ArrayList... Eu usei uma Matriz REAL para poder controlar o tamanho máximo do menu e para recuperar meus itens de menu sem ter que usar um molde.

As Implementações de Menus do Lou e Mel

```
public class DinerMenu {  
    static final int MAX_ITEMS = 6;  
    int numberOfItems = 0;  
    MenuItem[] menuItems;
```

← E aqui está a implementação de Mel para o menu do Restaurante.

← Mel utiliza uma abordagem diferente: ele está usando uma Matriz para poder controlar o tamanho máximo do menu e recuperar itens sem ter que colocar os objetos num molde.

```
public DinerMenu() {  
    menuItems = new MenuItem[MAX_ITEMS];  
    addItem("Vegetarian BLT",  
        "(Fakin') Bacon with lettuce & tomato on whole wheat", true, 2.99);  
    addItem("BLT",  
        "Bacon with lettuce & tomato on whole wheat", false, 2.99);  
    addItem("Soup of the day",  
        "Soup of the day, with a side of potato salad", false, 3.29);  
    addItem("Hotdog",  
        "A hot dog, with saurkraut, relish, onions, topped with cheese", false, 3.05);  
    // mais alguns itens de menu incluídos aqui  
}
```

← Como Lou, Mel também cria os itens de menu no construtor, usando o método auxiliar addItem().

← addItem() recebe todos os parâmetros necessários para criar um item de menu e gera uma instância de MenuItem. Além disso, ele verifica se o tamanho do menu não foi atingido.

As Implementações de Menus do Lou e Mel

```
    addItem("Hotdog",  
        "A hot dog, with saurkraut, relish, onions, topped with cheese", false, 3.05);  
    // mais alguns itens de menu incluídos aqui  
}  
public void addItem(String name, String description,  
    boolean vegetarian, double price)  
{  
    MenuItem menuItem = new MenuItem(name, description, vegetarian, price);  
    if (numberOfItems >= MAX_ITEMS) {  
        System.err.println("Sorry, menu is full! Can't add item to menu");  
    } else {  
        menuItems[numberOfItems] = menuItem;  
        numberOfItems = numberOfItems + 1;  
    }  
}  
public MenuItem[] getMenuItems() {  
    return menuItems;  
}
```

addItem() recebe todos os parâmetros necessários para criar um item de menu e gera uma instância de MenuItem. Além disso, ele verifica se o tamanho máximo do menu não foi atingido.

Mel deseja manter seu menu abaixo de um tamanho específico (presumivelmente, para não ter que se lembrar de muitas receitas).

getMenuItems() retorna a matriz de itens do menu.

Como Lou, Mel possui outros trechos de código que dependem do fato de que a implementação do menu é feita por uma

As Implementações de Menus do Lou e Mel

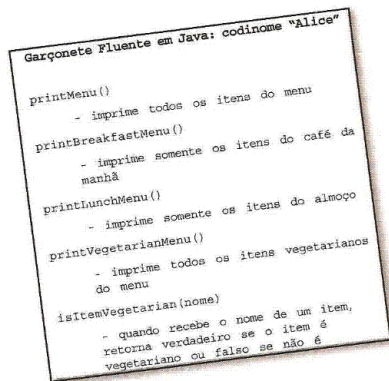
- Mel utiliza uma abordagem diferente: ele está usando uma Matriz para poder controlar o tamanho máximo e recuperar itens sem que ter colocar os objetos em um molde
- Como Lou, Mel também cria os itens de menu no construtor, usando o método auxiliar `addItem()`
- `addItem()` recebe todos os parâmetros necessários para criar um item de menu e gera uma instância de `MenuItem`. Além disso, ele verifica se o tamanho máximo do menu não foi atingido.
- Mel deseja manter seu menu abaixo de um tamanho específico (presumivelmente, para não ter que se lembrar de muitas receitas)

Qual é o problema em ter duas representações diferentes de menus?

- Para vermos como a existência de duas representações diferentes de menus complica as coisas, vamos implementar um cliente que utiliza os dois menus.
- Imagine que você tenha sido contratado pela nova companhia, formada pela fusão da Panquecaria com o Restaurante, para criar uma garçonete fluente em Java.
- As especificações dessa garçonete determinam que ela deva ser capaz de imprimir um menu personalizado para os fregueses que o solicitarem e também de dizer se um determinado item do menu é vegetariano sem ter que perguntar ao cozinheiro - isto, sim, é uma novidade!!

Especificações da Garçonete Fluente em Java

Vamos examinar as especificações e depois detalhar os passos necessários para implementá-las...



Esta garçonete está fazendo um curso de Java.



Especificações da Garçonete Fluente em Java

- 1 Para imprimir todos os itens de ambos os menus, você teria que chamar o método `getMenuItem()` em `PancakeHouseMenu` e `DinerMenu` para recuperar seus respectivos itens de menu. Observe que cada um retorna um tipo diferente:
 - A implementação está aparecendo: os itens do café da manhã estão numa lista matricial e os do almoço numa matriz.
 - Os métodos parecem idênticos, mas as chamadas estão retornando tipos diferentes.

Especificações da Garçonete Fluente em Java

```
1 PancakeHouseMenu pancakeHouseMenu = new
    PancakeHouseMenu();
2 ArrayList breakfastItems = pancakeHouseMenu.
    getMenuItems();
3
4 DinerMenu dinerMenu = new DinerMenu();
5 MenuItem[] lunchItems = dinerMenu.getMenuItems();
```

Especificações da Garçonete Fluente em Java

- ② Agora para imprimir os itens do menu da Panquecaria, faremos um loop através do ArrayList breakfastItems e para imprimir os itens do menu do Restaurante faremos um loop através da respectiva matriz.
 - Agora temos que implementar dois loops diferentes para recuperar individualmente os itens de menu de cada implementação.
 - Um loop para o ArrayList
 - Um outro loop para a matriz

Especificações da Garçonete Fluente em Java

```
1  for (int i = 0; i < breakfastItems.size(); i++) {
2      MenuItem menuItem = (MenuItem) breakfastItems.get(
3          i);
4      System.out.println(menuItem.getName());
5      System.out.println(menuItem.getPrice());
6      System.out.println(menuItem.getDescription());
7  }
8  for (int i = 0; i < lunchItems.length; i++) {
9      MenuItem menuItem = lunchItems[i];
10     System.out.println(menuItem.getName());
11     System.out.println(menuItem.getPrice());
12     System.out.println(menuItem.getDescription());
13 }
```

Especificações da Garçonete Fluente em Java

- 3 A implementação dos demais métodos na Garçonete será uma variação deste mesmo tema. Sempre precisaremos recuperar ambos os menus e usar dois loops para iterar através desses itens. Se outro restaurante, com outra implementação, for adquirido, teremos que fazer três loops.

E agora?

- Mel e Lou estão nos colocando numa situação difícil. Eles não querem mudar suas implementações, porque teriam que reescrever toneladas de código em suas respectivas classes de menus.
- Por outro lado, se ninguém ceder, teremos que implementar uma Garçonete muito difícil de manter e expandir.

E agora?

- Seria ótimo se pudéssemos encontrar alguma solução que lhes permitisse implementar a mesma interface para seus respectivos menus (que já são bem parecidos, com a exceção do tipo de retorno do método getMenuItems()).
- Assim, poderíamos minimizar as referências concretas no código Garçonete e, possivelmente, eliminar os múltiplos loops necessários para fazer a iteração através de ambos os menus.
- Parece tentador, não? Mas como é que vamos fazer isso?

Podemos encapsular a iteração?

- Devemos encapsular tudo aquilo que varia. O que está mudando aqui é evidente: a iteração, devido ao fato de que diferentes coleções de objetos são retornados pelos menus.
- Mas será que podemos encapsular isto? Vamos elaborar melhor essa ideia....

Podemos encapsular a iteração?

- 1 Para iterar através dos itens da panquecaria, usamos os métodos `size()` e `get()` na lista matricial

```
1 for (int i = 0; i < breakfastItems.size(); i++){  
2     MenuItem menuItem = (MenuItem) breakfastItems.  
        get(i);  
3 }
```

- 2 E, para iterar através dos itens do almoço, usamos o campo `length` e a notação de índice da matriz `MenuItem`.

```
1 for (int i = 0; i < lunchItems.length; i++){  
2     MenuItem menuItem = lunchItems[i];  
3 }
```

- 3 E se criássemos um objeto, que poderíamos chamar de `Iterator`, para encapsular a maneira como é feita a iteração, através de uma coleção de objetos?

Podemos encapsular a iteração?

Vamos testar essa abordagem com o ArrayList?

```
1 while(iterator.hasNext()){  
2     MenuItem menuItem = (MenuItem) iterator.next();  
3 }
```

- Pedimos a breakfastMenu um iterador de seus itens do menu
- O cliente simplesmente chama hasNext() e next(); por trás das cenas, o iterador chama get() na lista matricial

Podemos encapsular a iteração?

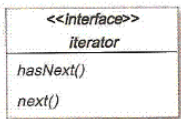
E, agora, vamos experimentar o mesmo procedimento com a Matriz:

```
1 Iterator iterator = lunchMenu.createIterator();  
2 while(iterator.hasNext()){  
3     MenuItem menuItem = (MenuItem) iterator.next();  
4 }
```

- Este código é exatamente igual ao que usamos com os itens de menu do café da manhã

Conheça o Padrão Iterator

- Bem, parece que nosso plano de encapsular a iteração pode realmente funcionar;
- E, como você talvez já tenha adivinhado, ele é um Padrão de Projetos conhecido como Padrão Iterator.
- A primeira coisa que você precisa saber sobre o Padrão Iterator é que ele depende de uma interface chamada Iterator.
- Aqui está uma interface Iterator possível:



O método `hasNext()` determina se há mais elementos no grupo para serem submetidos à iteração.

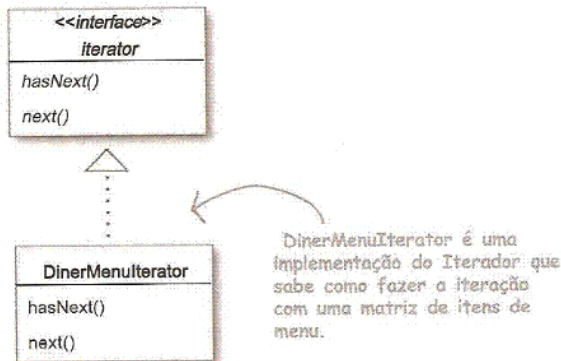
O método `next()` retorna o próximo objeto do grupo.

Conheça o Padrão Iterator

- Agora que já temos essa interface, podemos implementar Iteradores para qualquer tipo de coleção de objetos:
 - Matrizes
 - Listas
 - HashTables
 - ...
- Agora vamos dar um passo adiante, implementando esse Iterador e, conectando-o ao menu do restaurante para ver como funciona.

Conheça o Padrão Iterator

Escolha sua favorita. Digamos que você queira implementar o Iterator para a Matriz utilizada no menu do restaurante. Isto seria feita da seguinte maneira.



Acrescentando um Iterator ao Menu do Restaurante

Para acrescentar um Iterator ao menu restaurante, precisamos inicialmente definir a Interface do Iterator:

```
1 public interface Iterator {  
2     boolean hasNext();  
3     Object next();  
4 }
```

Acrescentando um Iterator ao Menu do Restaurante

E agora temos que implementar um Iterator concreto que funcione com o menu do restaurante:

```
public class DinerMenuIterator implements Iterator {  
    MenuItem[] items;  
    int position = 0;   
  
    public DinerMenuIterator(MenuItem[] items) {  
        this.items = items;  
    }  
  
    public Object next() {  
        MenuItem menuItem = items[position];  
        position = position + 1;  
        return menuItem;  
    }  
  
    public boolean hasNext() {  
        if (position >= items.length || items[position] == null) {  
            return false;  
        } else {  
            return true;  
        }  
    }  
}
```

"position" informa a posição atual da iteração dentro da matriz.

O construtor recebe a matriz de itens de menu que serão submetidos à iteração.

O método next() retorna o próximo item da matriz e incrementa a posição.

O método hasNext() verifica se já passamos por todos os elementos da matriz e retorna "verdadeiro" se ainda existem itens para serem submetidos à iteração.

Acrescentando um Iterator ao Menu do Restaurante

- **position** informa a posição atual da iteração dentro da matriz.
- O construtor recebe a matriz de itens de menu que serão submetidos à iteração
- O método next() retorna o próximo item da matriz e incrementa a posição
- O método hasNext() verifica se já passamos por todos os elementos da matriz e retorna "verdadeiro" se ainda existem itens para serem submetidos à iteração
- Como o dono do restaurante já havia alocado um tamanho máximo para a matriz, precisamos checar não apenas se já chegamos ao fim da matriz, mas também se o próximo item é nulo, o que indicaria que não existem mais itens
- Muito bem, agora temos o Iterator. Chegou a hora de encaixá-lo no menu do restaurante.

Incorporando o Iterator ao Menu do Restaurante

Tudo o que precisamos fazer é acrescentar um método para criar um DinerMenuIterator e retorná-lo ao cliente: oi

```
public class DinerMenu {  
    static final int MAX_ITEMS = 6;  
    int numberOfItems = 0;  
    MenuItem[] menuItems;  
    // construtor aqui  
    // addItem aqui  
public MenuItem[] getMenuItems() {  
    return menuItems;  
    }  
    public Iterator createIterator() {  
        return new DinerMenuIterator(menuItems);  
    }  
}
```

Incorporando o Iterator ao Menu do Restaurante

- Não precisamos mais do método `getMenuItems()`. De fato, queremos nos livrar dele, porque ele expõe nossa implementação interna!!
- O método `createIterator()` cria um `DinerMenuIterator` a partir da matriz `menuItems` e retorna ao cliente
- Estamos retornando a interface `Iterator`. O cliente não precisa saber como os itens do menu são mantidos no menu do restaurante. Ele só precisa saber os iteradores para acessar consecutivamente os itens do menu

Corrigindo o código da Garçonete

- Agora precisamos integrar o código do Iterator à Garçonete. Isto provavelmente nos permitirá reduzir parte da redundância.
- **A integração é bastante simples:** primeiro criamos um método `printMenu()` que aceita um Iterator, depois usamos o método `getIterator()` em cada menu para recuperar o Iterator e passá-lo para o novo método

Corrigindo o código da Garçonete

```
public class Waitress {  
    PancakeHouseMenu pancakeHouseMenu;  
    DinerMenu dinerMenu;  
  
    public Waitress(PancakeHouseMenu pancakeHouseMenu, DinerMenu dinerMenu) {  
        this.pancakeHouseMenu = pancakeHouseMenu;  
        this.dinerMenu = dinerMenu;  
    }  
  
    public void printMenu() {  
        Iterator pancakeIterator = pancakeHouseMenu.createIterator();  
        Iterator dinerIterator = dinerMenu.createIterator();  
        System.out.println("MENU\n----\nBREAKFAST");  
        printMenu(pancakeIterator);  
        System.out.println("\nLUNCH");  
        printMenu(dinerIterator);  
    }  
}
```

No construtor, a Garçonete recebe os dois menus.

Agora o método `printMenu()` cria dois iteradores, um para cada menu.

E então sobrecarrega `printMenu()` com cada iterador

Corrigindo o código da Garçonete

```
private void printMenu(Iterator iterator) {  
  
    while (iterator.hasNext()) {  
        MenuItem menuItem = (MenuItem)iterator.next();  
        System.out.print(menuItem.getName() + ", ");  
        System.out.print(menuItem.getPrice() + " -- ");  
        System.out.println(menuItem.getDescription());  
    }  
}
```



Teste se ainda
há outros itens.

Testando nosso código

Chegou a hora de avaliar o nosso trabalho. Vamos escrever um código de test-drive e ver como a Garçonete funciona....

```
public class MenuTestDrive {  
    public static void main(String args[]) {  
        PancakeHouseMenu pancakeHouseMenu = new PancakeHouseMenu();  
        DinerMenu dinerMenu = new DinerMenu();  
  
        Waitress waitress = new Waitress(pancakeHouseMenu, dinerMenu);  
  
        waitress.printMenu();  
    }  
}
```

The diagram illustrates the sequence of operations in the `MenuTestDrive` class. It features three handwritten annotations in Portuguese with arrows pointing to specific lines of code:

- An arrow points to the creation of `PancakeHouseMenu` and `DinerMenu` objects, with the annotation: "Primeiro criamos os novos menus." (First we create the new menus.)
- An arrow points to the creation of the `Waitress` object, passing the menu objects as arguments, with the annotation: "Depois criamos uma garçonete e passamos os menus para ela." (Then we create a waitress and pass the menus to her.)
- An arrow points to the `waitress.printMenu()` call, with the annotation: "Finalmente, exiba os menus." (Finally, display the menus.)

Testando nosso código

E aqui está o resultado...

File Edit Window Help GreenEggs@sam

```
% java DinerMenuTestDrive
```

```
MENU
```

```
----
```

```
BREAKFAST
```

```
K&B's Pancake Breakfast, 2.99 -- Pancakes with scrambled eggs, and toast  
Regular Pancake Breakfast, 2.99 -- Pancakes with fried eggs, sausage  
Blueberry Pancakes, 3.49 -- Pancakes made with fresh blueberries  
Waffles, 3.59 -- Waffles, with your choice of blueberries or strawberries
```

```
LUNCH
```

```
Vegetarian BLT, 2.99 -- (Fakin') Bacon with lettuce & tomato on whole wheat  
BLT, 2.99 -- Bacon with lettuce & tomato on whole wheat  
Soup of the day, 3.29 -- Soup of the day, with a side of potato salad  
Hotdog, 3.05 -- A hot dog, with saurkraut, relish, onions, topped with cheese  
Steamed Veggies and Brown Rice, 3.99 -- Steamed vegetables over brown rice  
Pasta, 3.89 -- Spaghetti with Marinara Sauce, and a slice of sourdough bread
```

```
%
```

Primeiro fizemos a
iteração através do
menu de panquecas.

Depois repetimos
o processo com o
menu do almoço,
usando sempre o
mesmo código de
iteração.

O que fizemos até agora?

- Para começar, nós deixamos nossos cozinheiros de Objetolândia muito felizes.
- Eles acertaram suas diferenças e mantiveram suas próprias implementações.
- Nesse processo, nós também nos beneficiamos, porque a Garçonete será muito mais fácil de manter e, mais adiante, expandir.

O que fizemos até agora?



O que fizemos até agora?

Implementação de Garçonete de Difícil Manutenção

Os Menus não são bem encapsulados; podemos ver que o restaurante está usando um `ArrayList` e a panquecaria, uma matriz.

Precisamos de dois loops para fazer a iteração através dos itens de menu.

A `Garçonete` está vinculada a classes concretas (`MenuItem[]` e `ArrayList`).

A `Garçonete` está vinculada a duas classes concretas de menus, embora suas interfaces sejam praticamente idênticas.

Garçonete Revista e Melhorada Usando um Iterator

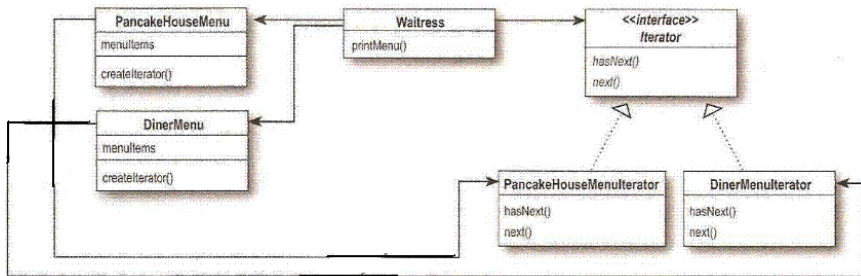
Agora as implementações dos menus estão encapsuladas. A `Garçonete` não tem a menor idéia de como os menus mantêm suas respectivas coleções de itens.

Precisamos de apenas um loop para lidar polimorficamente com qualquer coleção de itens, contanto que ela implemente o `Iterator`.

Agora a `Garçonete` usa uma interface (`Iterator`).

As interfaces dos menus agora são exatamente idênticas e... epa, ainda não temos uma interface comum, o que significa que a `Garçonete` continua dependendo de duas classes concretas de menus. É melhor consertarmos isso.

O que fizemos até agora?



O que fizemos até agora?

- O Iterator permite que a Garçonete seja desvinculada da forma com as classes concretas foram implementadas.
- Ela não precisa saber se o menu é implementado através de uma matriz, um ArrayList ou etc.
- Tudo que ela sabe é que pode obter um Iterador para fazer o trabalho de iteração

O que fizemos até agora?

- Agora estamos usando uma interface comum de Iterator e implementamos duas classes concretas
- PancakeHouseMenu e DinerMenu implementam o novo método `createIterator()`: eles são os responsáveis por criar o iterador para as respectivas implementações dos itens do menu
- Estes dois menus implementam exatamente o mesmo conjunto de métodos, mas não implementam a mesma interface. Teremos que corrigir isso para livrar a garçone de qualquer dependência de menus concretos.

O que fizemos até agora?

- Observe que o Iterator nos proporciona uma maneira de recuperar individualmente os elementos de um agregado, sem forçar esse agregado a congestionar a sua própria interface com um punhado de métodos que permitam a recuperação dos seus elementos.
- Além disso, a implementação do Iterator pode existir fora do agregado: em outras palavras, nós encapsulamos a iteração

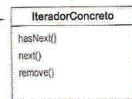
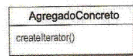
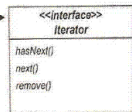
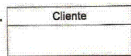
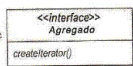
Definindo o Padrão Iterator

O **Padrão Iterator** fornece acessar sequencialmente os elementos de um objeto agregado sem expor a sua representação subjacente.

- Delega essa tarefa ao objeto iterador, não ao agregado, o que simplifica a interface e a implementação do agregado e coloca a responsabilidade onde ela deveria estar.

Definindo o Padrão Iterator

Ter uma interface comum para os seus agregados é prático para o seu cliente, porque o separa da implementação da sua coleção de objetos.



O AgregadoConcreto possui uma coleção de objetos e implementa o método que retorna um Iterator para a sua coleção.

Cada AgregadoConcreto é responsável pela criação de uma instância de IteradorConcreto que possa fazer a iteração através da sua própria coleção de objetos.

O IteradorConcreto é responsável pelo gerenciamento da posição atual da iteração.

Trabalho

- Muito bem, sabemos que as interfaces de PancakeHouseMenu e DinerMenu são exatamente idênticas, mas ainda não definimos uma interface comum para elas.
- Agora faremos isso refinar um pouco mais o funcionamento da Garçonete.

Trabalho

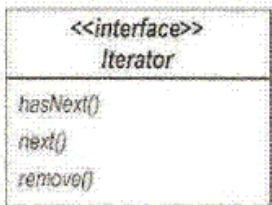
- Talvez você esteja se perguntando por que não estamos usando a interface `Iterator` do Java - bem, queríamos que você percebesse como é possível criar um iterador a partir de zero.
- Agora que já fizemos isso, vamos mudar para a interface `Iterator` do Java, porque ela é mais poderosa do que a nossa interface `Iterator` caseira.
- **Poderosa como? Logo você verá.**

Exercício 1

Adapte o exemplo para a interface `java.util.Iterator`

Trabalho

Em primeiro lugar, vamos conferir a interface `java.util.Iterator`:



Isto é muito parecido com a nossa definição anterior.

A diferença está neste método adicional, que nos permite remover o último item retornado pelo método `next()` a partir do agregado.

Trabalho

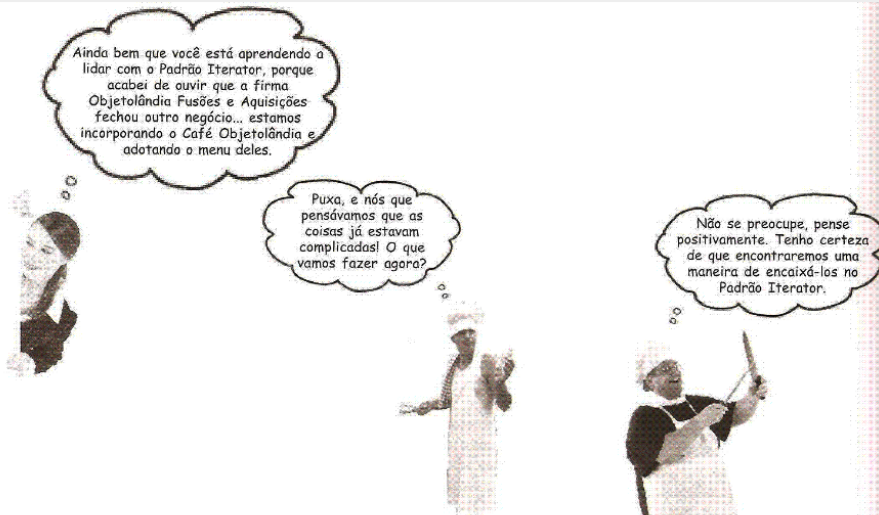
- Tudo que precisamos fazer é mudar a interface estendida por PancakeHouseMenuIterator e DinerMenuIterator, certo?
- Além de java.util possuir a sua própria interface Iterator, ArrayList também tem um método iterator() que retorna um iterador.
- Em outras palavras, na verdade nunca foi necessário que implementássemos o nosso próprio iterador para o ArrayList.

Trabalho

Observação

Por outro lado, nossa implementação continua a ser necessária em DinerMenu, porque ele é baseado numa matriz (Array), que não implementa o método `iterator()` (nem qualquer outra maneira de criar um iterador de matrizes)

Examinando o Menu do Café



Examinando o Menu do Café

Aqui está o menu do Café. Parece que não vai muito difícil integrá-lo à nossa estrutura. Vamos dar uma olhada:

```
public class CafeMenu {  
    Hashtable menuItems = new Hashtable();  
  
    public CafeMenu() {  
        addItem("Veggie Burger and Air Fries",  
            "Veggie burger on a whole wheat bun, lettuce, tomato, and fries",  
            true, 3.99);  
        addItem("Soup of the day",  
            "A cup of soup of the day, with a side salad",  
            false, 3.69);  
        addItem("Burrito",  
            "A large burrito, with whole pinto beans, salsa, guacamole",  
            true, 4.29);  
    }  
}
```

O menu do Café não implementa a nossa nova interface Menu, mas isso é fácil de corrigir.

O Café armazena seus itens de menu numa hashtable. Esse formato aceita o Iterator? É o que veremos a seguir...

Como nas outras menus, os itens de menu são inicializados no construtor.

Examinando o Menu do Café

Os Padrões Iterator e Composite

```
public void addItem(String name, String description,  
                    boolean Vegetarian, double price)  
{  
    MenuItem menuItem = new MenuItem(name, description, vegetarian, price);  
    menuItems.put(menuItem.getName(), menuItem);  
}  
public Hashtable getItems() {  
    return menuItems;  
}
```

É aqui que criamos um novo item de menu e o acrescentamos à hashtable MenuItem.

o valor do objeto menuItem.

a chave é o nome do item.

Não precisaremos mais disso.

Adaptando o código do menu do Café

- A integração do menu do Café à nossa estrutura é fácil. Porque Hashtable é uma das coleções em Java que comportam o Iterator.
- Mas não exatamente como ArrayList...

Hashtable é uma coleção obsoleta. Caso queira, mude a coleção.

Adaptando o código do menu do Café

```
public class CafeMenu implements Menu {
```

```
    Hashtable menuItems = new Hashtable();
```

```
    public CafeMenu() {
```

```
        //código do construtor aqui
```

```
    }
```

```
    public void addItem(String name, String description,
                        boolean vegetarian, double price)
```

```
    {
```

```
        MenuItem menuItem = new MenuItem(name, description, vegetarian, price);
```

```
        menuItems.put(menuItem.getName(), menuItem);
```

```
    }
```

```
    public Hashtable getItems() {
```

```
        return menuItems;
```

```
    }
```

```
    public Iterator createIterator() {
```

```
        return menuItems.values().iterator();
```

```
    }
```

O menu do Café agora implementa a interface Menu, que a Garçonete pode usar exatamente como nos dois outros menus.

Estamos usando Hashtable porque é uma estrutura comum de dados para armazenar valores; você também poderia usar uma estrutura mais recente, HashMap.

Como antes, podemos nos livrar de getItems() para não expor a implementação de menuItems à Garçonete.

E aqui é onde implementamos o método createIterator(). Observe que não estamos obtendo um Iterator para toda a Hashtable, mas apenas para os valores.

Adaptando o código do menu do Café



Close do código

A estrutura Hashtable é um pouco mais complexa que ArrayList, porque aceita simultaneamente chaves e valores, mas nada impede que obtenhamos um Iterator para os valores (que são os itens de menu).

```
public Iterator createIterator() {  
    return menuItems.values().iterator();  
}
```

Primeiro obtemos os valores da Hashtable, que formam uma coleção de todos os objetos que ela contém.

Felizmente, essa coleção aceita o método `iterator()`, que retorna um objeto do tipo `java.util.Iterator`.

Adaptando o código do menu do Café

Exercício 2

Adapte o exemplo para o cardápio de cafés.

Hashtable é uma coleção obsoleta. Caso queira, mude a coleção.

Iterator

Prof. Igor Avila Pereira
igor.pereira@riogrande.ifrs.edu.br

Instituto Federal de Educação, Ciência e Tecnologia do Rio Grande do Sul (IFRS)
Câmpus Rio Grande
Divisão de Computação